Самостійна робота

Тема : 2.

Виконав: Шалівський Віталій

Група: alk-43

Київ 2026 р

Хіт роботи:

```cpp
#include <iostream>
#include <string>
#include <windows.h>

using namespace std;

// ==================== БАЗОВИЙ КЛАС ====================
class LibraryItem {
protected:
    string title;

public:
    LibraryItem(const string& t) : title(t) {}

    virtual void PrintInfo() const {
        cout << "Елемент бібліотеки: " << title << "\n";
    }

    virtual ~LibraryItem() {}
};

// ==================== КЛАС КНИГА ====================
class Book : public LibraryItem {
    string author;

public:
    Book(const string& t, const string& a)
        : LibraryItem(t), author(a) {
    }

    void PrintInfo() const override {
        cout << "Книга: " << title
            << " | Автор: " << author << "\n";
    }
};
```

```cpp
// ==================== КЛАС ЖУРНАЛ ====================
class Magazine : public LibraryItem {
    int issueNumber;

public:
    Magazine(const string& t, int issue)
        : LibraryItem(t), issueNumber(issue) {
    }

    void PrintInfo() const override {
        cout << "Журнал: " << title
            << " | Номер випуску: " << issueNumber << "\n";
    }
};

// ==================== MAIN ====================
int main() {

    // Встановлення Windows-1251
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    // Створення об'єктів
    Book book("Кобзар", "Тарас Шевченко");
    Magazine magazine("Наука і життя", 12);

    // Поліморфізм через покажчик
    LibraryItem* ptr1 = &book;
    LibraryItem* ptr2 = &magazine;

    cout << "Виклик через покажчик (->):\n";
    ptr1->PrintInfo();
    ptr2->PrintInfo();
```

```cpp
    cout << "\n";

    // Поліморфізм через масив покажчиків
    LibraryItem* items[2];
    items[0] = &book;
    items[1] = &magazine;

    cout << "Вивід через масив покажчиків:\n";
    for (int i = 0; i < 2; i++) {
        items[i]->PrintInfo();
    }

    return 0;
}
```

Рис. 1 Каркас програми для демонстрації поліморфізму (Бібліотека)

На рисунку представлено загальну структуру програми. Оголошено базовий клас LibraryItem, похідні класи Book та Magazine, а також реалізовано виклики віртуальних методів. Каркас демонструє підготовку до перевірки механізму динамічного поліморфізму.

```
Виклик через покажчик (->):
Книга: Кобзар | Автор: Тарас Шевченко
Журнал: Наука і життя | Номер випуску: 12

Вивід через масив покажчиків:
Книга: Кобзар | Автор: Тарас Шевченко
Журнал: Наука і життя | Номер випуску: 12

D:\Desktop\студент\ООП\на 14 02 26\ооп Polimorphism__librari\x64\Debug\ооп Polimorphism__librari.exe (process 16672) exi
ted with code 0 (0x0).
Press any key to close this window . . .
```

Рис. 2 Результат запуску каркасу програми

На рисунку показано початковий запуск програми в середовищі Microsoft Visual Studio. Виводяться заголовки прикладів, що підтверджує коректну компіляцію та запуск програми.

Приклад 1. (2 класи і викликчерез покажчик ->)

```
#include <iostream>
#include <string>
#include <windows.h>

using namespace std;

// ================ БАЗОВИЙ КЛАС ================
class LibraryItem {
protected:
    string title;

public:
    LibraryItem(const string& t) : title(t) {}

    virtual void PrintInfo() const {
        cout << "Елемент бібліотеки: " << title << endl;
    }

    virtual ~LibraryItem() {}
};

// ================ ПОХІДНИЙ КЛАС ================
class Book : public LibraryItem {
private:
    string author;

public:
    Book(const string& t, const string& a)
        : LibraryItem(t), author(a) {
    }

    void PrintInfo() const override {
        cout << "Книга: " << title
             << ", Автор: " << author << endl;
    }
};
```

```
// ================ ПРИКЛАД ================
void Example1_TwoClasses() {
    cout << "=== Приклад: 2 класи, виклик віртуальної функції через покажчик (->) ===\n";

    LibraryItem baseItem("Загальний каталог");
    Book book("Кобзар", "Тарас Шевченко");

    LibraryItem* p1 = &baseItem; // покажчик на базовий
    LibraryItem* p2 = &book;     // базовий покажчик на похідний

    cout << "p1->PrintInfo(): ";
    p1->PrintInfo(); // LibraryItem::PrintInfo()

    cout << "p2->PrintInfo(): ";
    p2->PrintInfo(); // Book::PrintInfo() (поліморфізм)
}

int main() {
    // Встановлюємо кодування Windows-1251 для вводу/виводу консолі
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    Example1_TwoClasses();

    system("pause");
    return 0;
}
```

Рис. 3 – Поліморфізм для двох класів (виклик через покажчик)

На рисунку зображено реалізацію базового класу LibraryItem та похідного класу Book. Покажчик типу LibraryItem* посилається на об’єкт класу Book. Метод PrintInfo() оголошено як virtual, що забезпечує динамічне зв’язування.

```
D:\Desktop\студент\ООП\на
=== Приклад: 2 класи, виклик віртуальної функції через покажчик (->) ===
p1->PrintInfo(): Елемент бібліотеки: Загальний каталог
p2->PrintInfo(): Книга: Кобзар, Автор: Тарас Шевченко
Press any key to continue . . .
```

Рис. 4 – Результат виконання прикладу 1

На рисунку показано результат виконання першого прикладу. Незважаючи на те, що використовується покажчик базового класу, викликається реалізація методу похідного класу Book. Це підтверджує роботу поліморфізму.

Приклад 2. 3 класи і передача у функцію покажчика(*) на базовий клас

```cpp
#include <iostream>
#include <string>
#include <windows.h>

using namespace std;

// ================ БАЗОВИЙ КЛАС ================
class LibraryItem {
protected:
    string title;

public:
    LibraryItem(const string& t) : title(t) {}

    virtual void PrintInfo() const {
        cout << "Елемент бібліотеки: " << title << endl;
    }

    virtual ~LibraryItem() {}
};

// ================ ПОХІДНИЙ КЛАС: BOOK ================
class Book : public LibraryItem {
private:
    string author;

public:
    Book(const string& t, const string& a)
        : LibraryItem(t), author(a) {
    }

    void PrintInfo() const override {
        cout << "Книга: " << title
             << ", Автор: " << author << endl;
    }
};
```

```cpp
// ================ ПОХІДНИЙ КЛАС: MAGAZINE ================
class Magazine : public LibraryItem {
private:
    int issueNumber;

public:
    Magazine(const string& t, int num)
        : LibraryItem(t), issueNumber(num) {
    }

    void PrintInfo() const override {
        cout << "Журнал: " << title
             << ", Номер випуску: " << issueNumber << endl;
    }
};

// ========================================================================
// 2) ТРИ КЛАСИ: передача у функцію покажчика (*) на базовий клас
// ========================================================================

void PrintItemByPointer(LibraryItem* p) {
    cout << "[Функція PrintItemByPointer] ";
    p->PrintInfo(); // поліморфний виклик
}

void Example2_ThreeClasses_FunctionPointer() {

    cout << "============================\n";
    cout << "2) ТРИ КЛАСИ: LibraryItem -> Book, Magazine\n";
    cout << "   Передача у функцію покажчика (LibraryItem*)\n";
    cout << "============================\n";

    LibraryItem item("Картотека");
    Book book("Clean Code", "Robert C. Martin");
    Magazine mag("Наука і життя", 12);
```

```cpp
    PrintItemByPointer(&item);
    PrintItemByPointer(&book);
    PrintItemByPointer(&mag);

    cout << "\nАНАЛІЗ:\n";
    cout << "- Функція приймає LibraryItem*.\n";
    cout << "- Можна передати адресу будь-якого похідного класу.\n";
    cout << "- Завдяки virtual викликається правильна версія PrintInfo().\n\n";
}

// ================ MAIN ================
int main() {

    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    Example2_ThreeClasses_FunctionPointer();

    system("pause");
    return 0;
}
```

Рис. 5 – Поліморфізм для трьох класів (передача покажчика на базовий клас у функцію)

На рисунку зображено функцію, яка приймає параметр типу LibraryItem*. У функцію передаються об’єкти класів Book та Magazine. Завдяки віртуальному методу викликається відповідна реалізація функції PrintInfo().

```
D:\Desktop\студент\ООП\На
============================
2) ТРИ КЛАСИ: LibraryItem -> Book, Magazine
   Передача у функцію покажчика (LibraryItem*)
============================
[Функція PrintItemByPointer] Елемент бібліотеки: Картотека
[Функція PrintItemByPointer] Книга: Clean Code, Автор: Robert C. Martin
[Функція PrintItemByPointer] Журнал: Наука і життя, Номер випуску: 12

АНАЛІЗ:
- Функція приймає LibraryItem*.
- Можна передати адресу будь-якого похідного класу.
- Завдяки virtual викликається правильна версія PrintInfo().

Press any key to continue . . .
```

Рис. 6 – Результат виконання прикладу 2

На рисунку відображено результат виконання другого прикладу. Функція коректно працює з різними похідними класами, що підтверджує реалізацію поліморфізму при передачі покажчика у функцію.

Приклад 3. 3 класи і виклик віртуальної функції у методі передача посилання (&) на базовий клас

```
#include <iostream>
#include <string>
#include <windows.h>

using namespace std;

// ================ БАЗОВИЙ КЛАС ================
class LibraryItem {
protected:
    string title;

public:
    LibraryItem(const string& t) : title(t) {}

    virtual void PrintInfo() const {
        cout << "Елемент бібліотеки: " << title << endl;
    }

    virtual ~LibraryItem() {}
};

// ================ ПОХІДНИЙ КЛАС: BOOK ================
class Book : public LibraryItem {
private:
    string author;

public:
    Book(const string& t, const string& a)
        : LibraryItem(t), author(a) {
    }

    void PrintInfo() const override {
        cout << "Книга: " << title
             << ", Автор: " << author << endl;
    }
};
```

```cpp
// ================ ПОХІДНИЙ КЛАС: MAGAZINE ================
class Magazine : public LibraryItem {
private:
    int issueNumber;

public:
    Magazine(const string& t, int num)
        : LibraryItem(t), issueNumber(num) {
    }

    void PrintInfo() const override {
        cout << "Журнал: " << title
            << ", Номер випуску: " << issueNumber << endl;
    }
};

// ============================================================================
// 3) ТРИ КЛАСИ: виклик віртуальної функції у методі
//    Передача посилання (&) на базовий клас
// ============================================================================

class LibraryPrinter {
public:
    void PrintByReference(const LibraryItem& item) {
        cout << "[Метод LibraryPrinter] ";
        item.PrintInfo(); // поліморфний виклик
    }
};

void Example3_ThreeClasses_MethodReference() {

    cout << "============================\n";
    cout << "3) ТРИ КЛАСИ: LibraryItem -> Book, Magazine\n";
    cout << "   Передача посилання (LibraryItem&)\n";
    cout << "   Виклик віртуальної функції у методі\n";
    cout << "============================\n";
```

```cpp
    LibraryPrinter printer;

    LibraryItem item("Архів");
    Book book("C++ Primer", "Lippman");
    Magazine mag("IT World", 5);

    printer.PrintByReference(item);
    printer.PrintByReference(book);
    printer.PrintByReference(mag);

    cout << "\nАНАЛІЗ:\n";
    cout << "- Метод приймає const LibraryItem&.\n";
    cout << "- Передається посилання, а не копія.\n";
    cout << "- Завдяки virtual працює динамічний поліморфізм.\n";
    cout << "- Викликається відповідна версія PrintInfo().\n\n";
}

// ================ MAIN ================
int main() {

    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    Example3_ThreeClasses_MethodReference();

    system("pause");
    return 0;
}
```

Рис. 7 – Поліморфізм для трьох класів (передача посилання на базовий клас у метод)

На рисунку показано метод класу Librarian, який приймає параметр типу LibraryItem&. Усередині методу викликається віртуальна функція PrintInfo(), що забезпечує поліморфну поведінку.

```
D:\Desktop\студент\ООП\На

============================
3) ТРИ КЛАСИ: LibraryItem -> Book, Magazine
   Передача посилання (LibraryItem&)
   Виклик віртуальної функції у методі
============================
[Метод LibraryPrinter] Елемент бібліотеки: Архів
[Метод LibraryPrinter] Книга: C++ Primer, Автор: Lippman
[Метод LibraryPrinter] Журнал: IT World, Номер випуску: 5

АНАЛІЗ:
- Метод приймає const LibraryItem&.
- Передається посилання, а не копія.
- Завдяки virtual працює динамічний поліморфізм.
- Викликається відповідна версія PrintInfo().

Press any key to continue . . .
```

Рис. 8 – Результат виконання прикладу 3

На рисунку показано результат роботи третього прикладу. Метод класу Librarian коректно обробляє різні типи об’єктів (Book, Magazine) через посилання на базовий клас. Це демонструє роботу поліморфізму при передачі параметра за посиланням.

У роботі реалізовано механізм динамічного поліморфізму в мові C++. Використання віртуальних функцій дозволяє працювати з різними типами об’єктів через єдиний інтерфейс базового класу.

Поліморфізм підвищує гнучкість програмної системи «Бібліотека» та спрощує її розширення новими типами елементів (наприклад, електронна книга, аудіокнига тощо).